Verifiable Chain-of-Custody for Digital Evidence Files

A Standalone Hash-Chain and Merkle-Tree Approach Without Distributed Consensus

Chilakapati Srijaya Chakradhar
Dept. of Computer Science and Engineering
KLH University (Bowrampet)
Hyderabad, India
2320030360@klh.edu.in

Gogineni Nikhil Sai
Dept. of Computer Science and Engineering
KLH University (Bowrampet)
Hyderabad, India
2320030195@klh.edu.in

Pusunuru Preetham
Dept. of Computer Science and Engineering
KLH University (Bowrampet)
Hyderabad, India
2320030438@klh.edu.in

Dr. Archana Kalidindi (Supervisor)
Dept. of Computer Science and Engineering
KLH University (Bowrampet)
Hyderabad, India

Abstract— Most forensic labs keep chain of custody by procedure, which does not let an outside party check later that a stored file or its custody record was left alone. Blockchain custody systems allow that check, but bring a ledger, peer nodes and consensus that a lab that is sole custodian of its own evidence does not need. We show that, for such a lab, the transparency-log toolkit (hash-linked signed entries, an RFC 6962 Merkle tree, consistency proofs and witness cosignatures) gives the properties the blockchain custody literature wants, without a ledger. None of these mechanisms is new; we apply them to forensic chain of custody as a working system and evaluate what it does and does not stop. Each file is hashed with SHA-256 into the Merkle tree, each custody action is a hash-linked entry signed with ECDSA, and a separate verifier rechecks both. Because a hash chain alone cannot see truncation or a rewrite by the key holder, the custodian signs checkpoints that independent witnesses cosign only after a consistency proof shows they extend the last one they signed. Of 12 attack scenarios on synthetic data, 8 were stopped, including a stolen-key rewrite and an unanchored truncation; the 4 that were not are the design's limits, chiefly that a witness protects only what it has already cosigned. A checkpoint is about 500 bytes, and a consistency proof over 100,000 entries is 14 hashes, checked in about 17 µs. Process-pool hashing registers a 10,000-file, 625 MiB folder in 0.42 s.

Keywords— *chain of custody, digital forensics, Merkle tree, hash chain, ECDSA, tamper-evident logging, log anchoring*

I. INTRODUCTION

Chain of custody is the record of who collected a piece of evidence, who handled it afterwards, and when. If that record cannot be defended, the file it describes loses much of its weight in court, however incriminating the contents are.

Most digital forensics labs manage custody with procedure. The procedure is reasonable, but it has two weak points. First, nothing in it allows an outsider to confirm later that a stored file or its log entry was not quietly changed; the assurance is only as good as the people who kept the records. Second, when someone does suspect tampering, a logbook gives no way to say *where* the record broke. The usual answer is a manual audit of every entry.

A line of recent work, which we review in Section III, answers both problems with a blockchain, typically a private Ethereum network or a Hyperledger Composer deployment. The tamper-evidence they get is real. So is the cost: nodes to run, a ledger to maintain and a consensus protocol to configure. Consensus exists so that several parties who do not trust each other can agree on a single history. A forensic lab that is the sole custodian of its evidence has one history and nobody competing to rewrite it.

So we asked which parts of a blockchain do the tamper-evidence work, and whether they can be used on their own. The answer is older than blockchain. Hash-linked timestamps [1], hash-chained audit logs written for computer forensics [2], Merkle history trees with consistency proofs [3] and, more recently, transparency logs checked by independent witnesses [4], [5] all give tamper evidence without a ledger. A recent survey of chain-of-custody practice [6] finds the academic work concentrated on blockchain infrastructure, rates system-oriented tools low on verifiability, and calls for tailored, lighter systems. This paper applies the transparency-log toolkit to that gap, and reports the system after two rounds of work, which changed more than we expected. Our contributions are:

1. **The application, and the gap it fills.** A custody design that carries the transparency-log mechanisms over to a single custodian: hash-linked signed entries, an RFC 6962 Merkle tree, an exported anchor, and witness-cosigned checkpoints with consistency proofs, with an explicit threat model, as a working open system (Sections IV and V). None of the mechanisms is new; in the sources we could reach we found no prior forensic custody system that combines them (Section III).
2. **An evaluation that includes what the design does not stop.** On synthetic data, a hash chain alone misses three of seven attacks; witnessed checkpoints stop 8 of 12 attack scenarios, and the 4 they do not stop are reported as limits, together with an analysis of whose trust the design moves and where (Sections VI-B, VI-H and VII).
3. **Lessons from building it.** The flaws we found in our own earlier design (leaf and node confusion and odd-node duplication in the Merkle tree, an unnamed SHA-1 inside the signatures), a benchmark that was several times too

slow because of a per-file buffer allocation, and a machine slowdown that distorted timings (Sections IV-C, IV-D and VI-C).

We also describe design choices that are standard practice but worth recording: a verifier that shares no code with the logger (Section V-B), a single append-only SQLite store, multi-evidence custody events, nested Merkle proofs for folders, and process-pool hashing (Sections V-C to V-F). We do not claim them as contributions.

II. MOTIVATION

Three observations from the literature survey shaped the project. The first is not surprising: manual, logbook-based custody is slow, error-prone and hard to audit [7]. The second is that the blockchain custody systems we reviewed [8]–[10] achieve real cryptographic tamper-evidence, but each one pays for it with ledger infrastructure, whether or not the deployment has several distrusting parties. The third comes from outside forensics. Work on tamper-evident logging [3], [11] shows that Merkle-tree logs are practical without any ledger, but it targets generic logs such as IoT telemetry, where there is no investigator whose signature has to be attributed to each action.

Read together, these describe a gap, not a contradiction. The non-blockchain logging work has the right cost but no accountable actor. The blockchain custody work has accountable actors (through signatures tied to identities) but carries a consensus layer that a single-custodian lab does not need. We take the techniques the logging literature has already validated, add signatures for attribution, and leave out consensus.

III. RELATED WORK

We reviewed ten sources; the fuller annotated summaries are in the project's companion document. Here is what each contributes to our argument.

- **Bonomi, Casini and Ciccotelli [8]** present B-CoC, a private-Ethereum chain of custody with a prototype and latency and storage measurements. It is the closest prior system in spirit, and it needs a running blockchain network.
- **Yagiz, Horasan and Yurttakal [11]** describe a Merkle-tree pipeline with adaptive chunking for IoT log integrity, at more than 130,000 entries per second and with no ledger. Technically it is the closest match to our approach, but it targets telemetry, not forensic evidence.
- **Crosby and Wallach [3]** introduced history trees: Merkle-tree-structured logs with O(log n) inclusion and consistency proofs. Our Merkle module follows their inclusion-proof construction.
- **Lone and Mir [9]** propose Forensic-chain on Hyperledger Composer, using hash pointers to keep an append-only evidence trail.
- **Ahmad, Khanji, Iqbal and Kamoun [10]** extend a private-Ethereum custody chain to physical evidence with smart locks, adding an IoT hardware dependency to the ledger one.
- **Patil, Kohli, Puri and Puri [7]** review blockchain for custody records. They frame the problem well but do not prescribe a design.
- **Al-Khateeb, Epiphaniou and Daly [12]** argue that evidentiary integrity should be shown through a verifiable record that can be challenged, which is the admissibility case for cryptographic verifiability in general.
- **Prakash and Sadawarti [13]** combine RC4 encryption with fuzzy-hash segmentation. Their emphasis is on encrypting the evidence, which differs from our focus on tamper-evident event chaining.
- **Miller and Singh [14]** record custody events as signed, replay-resistant on-chain transactions using smart contracts and wallets. This supports our view that signed events, and not hashes alone, are what a custody record needs, although they realise it with contracts and wallets and not a standalone signer.
- **Sakshi, Malik and Sharma [15]** use a block-header layout (Merkle root, previous hash, timestamp) for IoT multimedia evidence, which shows the structure we use is already accepted in the forensic literature.

One reference needed correcting. During a citation check we found that Miller and Singh [14] had been recorded in an earlier summary as "Chain of Custody and Evidence Integrity Verification Using Smart Contracts" (2021). The verified title is "Chain of Custody and Evidence Integrity Verification Using Blockchain Technology", ICCWS 2024, and the reference list below reflects that.

Four further lines of work matter for what we can claim.

Tamper-evident logging. Hash-linked timestamps [1] and audit logs whose entries cannot be undetectably modified or destroyed after a compromise, written for computer forensics [2], gave tamper evidence without any ledger long before blockchain, and a digital chain of custody kept and validated as a hash chain was described in 2007 [16]. Crosby and Wallach [3], already listed above, added logarithmic membership and consistency proofs. The limit our *anchor* addresses is a known one: practitioners document that a bare hash chain cannot show that its tail was deleted, and that records after the last anchor stay unprotected [17]. A recent forensic acquisition system for radio evidence detects truncation for the same reason, with periodic checkpoints over its hash chains [18].

Custody surveys and evidence containers. A recent survey of chain-of-custody practice [6] groups the field into paper trails, system-oriented tools and infrastructure-driven designs, mostly blockchain. It finds the infrastructure-driven designs strongest on verifiability but heavy and little adopted, and it does not discuss transparency logs or witnesses. On the container side, ECo-Bag [19] builds an evidence bag on a Merkle tree, Horodocs [20] timestamps evidence by batching its hashes into a Merkle tree whose root is recorded on Ethereum, and an extension of the AFF4 forensic format that stores Merkle hashes has been described [21]. All three address the integrity of the evidence itself. In the material we could read we found no signed custody log checked by independent witnesses in any of them.

Transparency logs and witnesses. Certificate Transparency [4], [22] publishes signed Merkle-tree heads for public logs so that independent monitors can detect a log that rewrites or forks its history. Transparency-log ecosystems add *witnesses*: independent parties that cosign a log's checkpoint only after checking, with a consistency proof, that it extends the last checkpoint they signed [23], [24]. A single witness is enough to show that a log is append-only, and several make it hard to show different histories to different parties. The idea that a diverse group of witnesses should validate and log a statement before anyone accepts it comes from decentralized witness cosigning [5]. Sigsum builds a witnessed log of signed checksums whose stated threat model includes an attacker who controls the signer's key, the log's key and a threshold of witnesses [25], which is the threat model we adopt. The IETF's SCITT drafts generalise the same idea to signed statements registered with transparency services [26].

What is, and is not, new here. The mechanism of Section V-H is therefore not ours. It follows the rule that Sigsum and the C2SP specification define for witnesses (cosign a checkpoint only if it extends the last one signed), applied to a custody log and run with files instead of a network, with our own file format and keys rather than their wire format. The application is the contribution: in the ten custody papers we reviewed, in the survey above, and in the forensic systems that turned up in our searches, we found no system that has a custodian's log checked by independent witnesses. The closest forensic work we found is the radio-evidence system above [18]. Its checkpoints carry an RFC 3161 timestamp, which shows *when* a checkpoint existed but not that it extends the previous one: a timestamp authority would timestamp two different histories of the same log without objection, where a witness refuses the second. It uses no Merkle tree, consistency proofs or witnesses. A timestamp and a witness answer different questions, and Section VIII lists a timestamp on each of our checkpoints as future work. That is a statement about our search, not proof that no one has done it. The search used a general web search and the OpenAlex and arXiv indexes, which cover titles and abstracts, not full-text searches of the major publisher databases. One commercial platform advertises signed custody records and trusted timestamps, and another advertises external anchoring, but we could not verify either design from public documentation. Related work also appears quickly: during this project a design issue for witnessed receipts in a software audit log and a preprint on Merkle-linked case evidence for agent workflows both appeared, neither about forensic custody.

IV. METHODOLOGY AND SYSTEM ARCHITECTURE

A blockchain combines several ingredients, most of which predate it (Section III). The system keeps the ones that matter for a single custodian and leaves out the rest. Table I lists what was kept and why.

Ingredient	Kept?	Role in this system
SHA-256 hashing	Yes	Fingerprints every evidence file; any changed byte shows up
Hash-linking	Yes	Each entry holds the previous entry's hash, so editing entry *N* breaks every later link
Merkle tree	Yes	Shows one file belongs to the registered set without re-hashing the others
ECDSA signatures	Yes	Ties each custody action to a key holder; forging an entry requires the private key
Distributed consensus / mining	No	Settles disputes between several parties; there is only one custodian here
Public or permissionless ledger	No	Confidentiality of evidence is a requirement, not a drawback to avoid

Table I. Blockchain ingredients kept and omitted.

A. System Architecture

Fig. 1 shows the system and the three kinds of machine it spans. Evidence passes through seven stages, numbered in the figure: intake, SHA-256 hashing, Merkle construction, custody logging, ECDSA signing, storage, and independent verification. Each stage has its own module (`hashing.py`, `merkle.py`, `hash_chain.py`, `ecdsa_signer.py`, `db.py`) and the modules do not depend on each other in a cycle. That separation is what lets the verifier rely on only two primitives, hashing and signature checking. The signing key and the database stay on the custodian's machine. What leaves it is a set of files: the exported log and evidence index, the anchor (Section V-D), and checkpoints with their consistency proofs (Section V-H). The auditor and the witnesses work only from those files and the custodian's public key, which reaches them by another route.

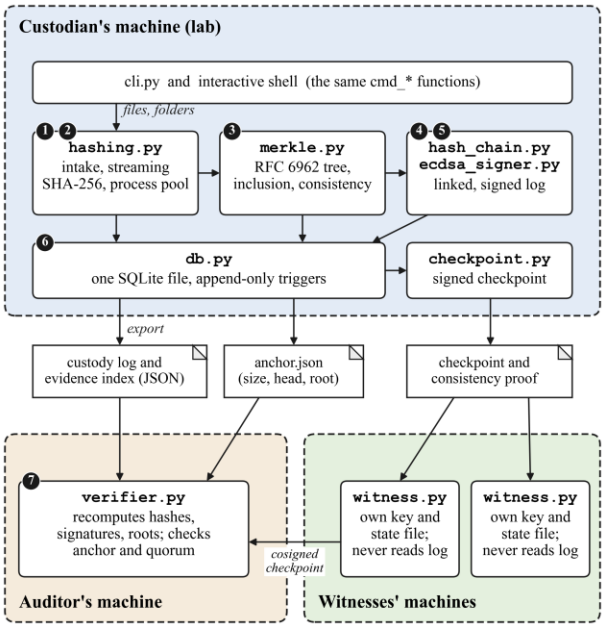


Fig. 1. System architecture. Dashed outlines are separate machines; numbers mark the seven stages of Section IV-A. Only files cross between machines.

B. Hash-Chain Custody Log

An event holds an index, evidence identifier(s), actor, action, timestamp and `prev_hash` (Fig. 2). The `entry_hash` is the SHA-256 of a canonical serialisation of those fields, and `prev_hash` is the previous entry's `entry_hash` (or a fixed genesis value for the first entry). This is the block-header idea applied to a single, linear, single-writer log. The link still makes a change to any past entry detectable, because no

network of peers is needed to decide which chain is the real one when there is only one.

Two serialisations exist, and both are covered by tests. Single-evidence entries use a delimited string. Multi-evidence entries (Section V-E) use sorted-key JSON, which avoids a delimiter-collision bug we found in an earlier version, where field values containing the separator could produce the same payload from different fields.

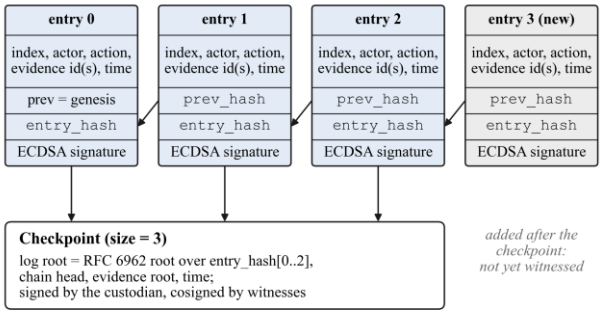


Fig. 2. Custody log. Each entry's prev_hash is the previous entry's entry_hash, and each entry_hash is signed. A checkpoint (Section V-H) commits to the first size entries; an entry added after it is not yet witnessed.

C. Merkle Tree over Evidence

Each registered file's digest is a leaf. The tree follows RFC 6962 [4]: a leaf is hashed as SHA-256(0x00 ‖ digest), an inner node as SHA-256(0x01 ‖ left ‖ right), and a lone node at the end of a level moves up unchanged (Fig. 3). An inclusion proof has at most ceil($\log_2 n$) steps and shows that one file's digest is part of the committed set.

The first version hashed leaves and nodes in the same way and paired an odd node with itself. We found two flaws in it and reproduced both in our own code. The roots of [a, b, c] and [a, b, c, c] were equal, the shape behind Bitcoin's CVE-2012-2459. And a 64-byte file holding two leaf digests hashed to the node above them, so it verified as a leaf with a shorter proof. The prefixes and the moved-up node remove both. A test written from the RFC text checks every tree size up to 40 against four implementations (the tree, the streaming builder, the streaming proof generator and the verifier's own copy). A database made under the old rules is refused until `cli.py migrate-merkle` converts it, which first checks each stored folder root against its members, so that the conversion cannot launder a folder that was altered.

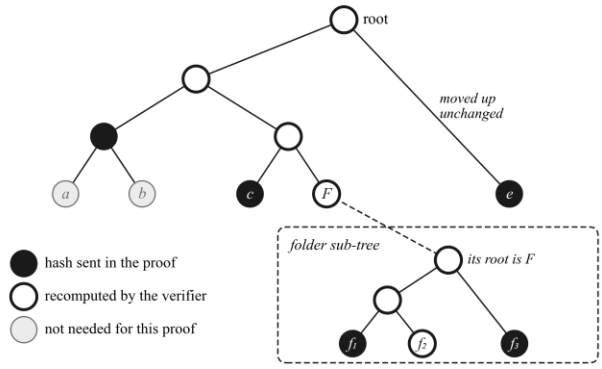


Fig. 3. RFC 6962 tree over evidence digests. A folder is one leaf, F, whose value is the root of its own sub-tree (Section V-F), so the proof for file f_2 has two hops: f_2 to F, then F to the root. Leaves are hashed as SHA-256(0x00 ‖ digest), inner nodes as SHA-256(0x01 ‖ left ‖ right).

D. ECDSA Signing

Every `entry_hash` is signed with a secp256k1 private key held by the custodian, and the signature is stored with the entry. The message is hashed with SHA-256 before signing. The first version did not name a hash, and the library's default is SHA-1, which NIST does not allow for signature generation; the signed text was a SHA-256 hex string, so we know of no practical attack, but it was a weak link in what we called a SHA-256 design. Entries signed that way still verify and are reported as legacy, and the verifier can be told to refuse them. In our development measurements, signing took about 0.5 ms and verification about 1.6 ms per entry.

E. Threat Model

In scope: a single-custodian lab with one system of record; after-the-fact edits to a custody entry or an evidence file; removal, reordering or truncation of log entries; and rewriting of history by someone who has the log but not the signing key. With a checkpoint cosigned by independent witnesses (Section V-H), we also treat a rewrite or truncation by someone who has the signing key as in scope, for everything the witnesses had already cosigned, provided fewer than a quorum of the witness keys are compromised too. An anchor kept outside the database gives a weaker form of the same protection that rests on the custodian alone.

Out of scope: Byzantine or multi-party disagreement, which is what consensus is for; real-time prevention of edits by someone who already has write access (we detect, we do not prevent); entries added after the last cosigned checkpoint (no witness has seen them yet); a log rewritten before any witness has cosigned it (a witness trusts a log on first sight); compromise of the signing key and a whole quorum of witness keys; and confidentiality of evidence content at rest. The previous version encrypted its SQLite store with AES-256-GCM. The current store does not (Section V-C), and we explain why there.

V. IMPLEMENTATION

The system is written in Python 3.11 with a small dependency set: `ecdsa` for signatures, `prompt_toolkit` for the interactive shell, `cryptography` only for the optional AES-GCM experiment, and otherwise the standard library (`hashlib`, `sqlite3`, `argparse`, `json`).

A. Module Structure

Module	Responsibility
vcoc/hashing.py	Streaming SHA-256; process-pool hashing in batches
vcoc/merkle.py	RFC 6962 tree, inclusion and consistency proofs, two-hop nested proofs
vcoc/checkpoint.py	Signed checkpoint of the custody log; consistency proof since an earlier size
vcoc/hash_chain.py	Append-only, hash-linked, ECDSA-signed custody log
vcoc/ecdsa_signer.py	Key generation, signing, verification
vcoc/db.py	Single SQLite store: custody_log, evidence, folders; append-only triggers
vcoc/models.py	Evidence and custody-event types; both event shapes
vcoc/interactive/	Menu-driven shell, live search, verification and checkpoint screens, status header
cli.py	One-shot commands, including export (the anchor), checkpoint and migrate-merkle
witness.py	Standalone witness: cosigns a checkpoint only if it extends the last one; imports the verifier
verifier.py	Standalone verifier; imports nothing from vcoc
scripts/	Experiment harnesses: scale, tamper (basic and extended), storage, witness scenarios and costs

Table II. Modules and what each one does.

B. Independence of the Verifier

verifier.py runs as its own process, reads only what an outside auditor would have (the exported JSON files and the public key), and rebuilds every check with hashlib and ecdsa. It does not import vcoc.hash_chain or vcoc.merkle. Even the hashing helpers it uses for large files are its own copy. The consequence is that a defect or a backdoor in the logger cannot cause the verifier to approve a tampered log, because the two code paths do not meet. The price is duplicated logic: the nested-proof check and the parallel re-hash exist twice, and a format change has to be made in both places. We think that is the right trade, and the tests check agreement between the two implementations on both event shapes and on mixed solo-file and folder evidence.

C. A Single Append-Only SQLite Store

The first version kept a JSON log and an encrypted SQLite mirror. We replaced both with one SQLite file holding plain columns, because plain columns can be queried, joined and exported without decrypting anything, and because the integrity guarantees come from the hashes and signatures, not from the storage layer. Two triggers on custody_log abort any UPDATE or DELETE. These protect against bugs and careless edits through the application. They are *not* an integrity guarantee: anyone with write access to the file can drop a trigger. The hash chain, the signatures and the anchor are what protect against that. The export command regenerates the JSON files the verifier reads.

The AES-256-GCM EncryptedStore is still in the repository because the storage experiment in Section VI-D uses it, but the CLI and shell no longer call it.

D. Anchoring and Truncation Detection

A hash chain checks itself from the start. It cannot tell that the last entries were deleted, because what remains is still a valid chain. It also cannot tell that someone with the signing key rewrote the log from some point onward. Both gaps are closed by recording, outside the database, two short values. cli.py export writes them to anchor.json: the number of custody entries with the head entry's hash, and the Merkle root with its leaf count.

The verifier, given --anchor anchor.json, checks that the entry at the anchored position still has the anchored hash. Because each hash covers the previous one, that single comparison pins every earlier entry. A log that has only *grown* since the anchor passes. With --evidence-index it also recomputes the Merkle root and compares it. The interactive shell keeps a weaker, local version of the same idea: it remembers the largest number of entries it has verified and shows SHORTENED if the log later has fewer. That value lives in shell_config.json on the same machine, so we treat the header as a convenience, and the anchor is the authoritative check.

The anchor shifts a trust requirement; it does not remove it. Someone must keep the copy somewhere the custodian cannot edit: a case ticket, an email to the supervisor, a printout.

E. Multi-Evidence Custody Events

One physical action, such as seizing a laptop and its external drive, usually concerns several evidence items. A custody event can now list several evidence identifiers, plus an optional case number, tag and notes, under one signature and one chain entry. Existing single-evidence entries are unchanged and still verify.

F. Folder Registration, Nested Proofs and Parallel Hashing

Registering a seized disk image by hand, file by file, does not scale. Pointing add-evidence at a folder walks it recursively, skips symlinks and special files, hashes each file, and registers it as {folder_id}/{relative_path}. The folder's files form their own Merkle sub-tree, and that sub-tree's root becomes one leaf in the main tree. The proof for a file inside a folder therefore has two hops: file to sub-tree root, then sub-tree root to main root (Fig. 3). A file registered on its own is the same format with an empty second hop.

Four changes made folder ingest and re-hash practical at large sizes.

- **Streaming and SQL search.** The efficiency pass changed ingest to stream files through the hasher, and evidence search to run as SQL queries instead of loading every row.
- **A cheaper read path.** Reading 1 MiB per call allocated a 1 MiB buffer for every tiny file. In a development sample of 9,288 real small files, one 64 KiB read for small files and a reused 1 MiB buffer only for larger ones cut hashing from roughly 0.9–1.0 ms to 0.14 ms per file in a single process, with identical digests. A plain 64 KiB chunk size would have been about 40% slower on a 1 GiB file, which is why the read is hybrid.

- **One reused read buffer.** The hybrid read still had a flaw. Any file of 64 KiB or more took the second path, which allocated a fresh 1 MiB buffer for every file. That allocation cost about 0.22 ms, and hashing 64 KiB takes only about 0.03 ms. The cost per file jumped from 0.060 ms at 65,535 bytes to 0.289 ms at 65,536 bytes, one byte more, and our synthetic benchmark files were exactly 64 KiB, the worst case. We found this while explaining an unexpectedly slow benchmark (Section VI-C). The buffer is now kept per thread and reused. It is taken out of its slot while a hash runs, so a nested call from a progress callback gets its own buffer, and it is put back even if the callback raises. The verifier's separate copy of the code received the same change.
- **A process pool.** In a development study on 23,218 files, threads gave only 1.4× (the interpreter lock limits them), while processes gave 6.3× at four workers and 9.8× at eight. Those ratios predate the buffer fix and were measured against a slower single-process baseline, so they overstate what the pool adds today (Section VI-E). The pool works in batches of 500 files or 64 MiB, files of 256 MiB or more are hashed in the caller so their progress stays visible, and the pool is used only from 2,000 files up, because below that its start-up costs more than it saves. The default is half the CPUs, capped at eight (each worker uses about 20–24 MiB), and `VCOC_HASH_JOBS` or `--jobs` overrides it. Results are identical to single-process hashing.

G. Interactive Shell

The shell wraps every one-shot command in an arrow-key menu with live search and path autofill. It calls the same `cmd_*` functions as the CLI, so the two interfaces cannot drift apart. It shows a status line with the last chain verification (`CLEAN`, `STALE`, `TAMPER`, `SHORTENED`) and the last evidence check, and it never runs a verification by itself. The Evidence screen sorts files into categories (all green, tampered, not logged, unverifiable) so that "no custody event mentions this file" is reported separately from "this file no longer matches its hash". Reading the status costs about 2 ms the first time and under 0.1 ms afterwards on a database of 92,879 evidence rows.

Two menu items cover the custodian's side of witnessing (Section V-H). *Write checkpoint for witnesses* signs a checkpoint, with the consistency proof from the size the witnesses last cosigned, and prints the `witness.py` command each witness runs on their own machine. *Verify against cosigned checkpoint* checks the log against the file the witnesses send back, with the verifier's rules and a quorum of trusted witness keys. The shell does not cosign: the custodian's own tool must not approve its own log. On a terminal tall enough for it, the header adds a witness line built from the last recorded check (all entries cosigned, *n* of *m* entries cosigned, or a failure), again without verifying anything on redraw. The line says how many entries are not yet covered, because those are the ones a witness does not protect.

H. Witness-Cosigned Checkpoints

This section adapts the witness rule of Sigsum and C2SP (Section III) to a custody log. A *checkpoint* commits to the custody log at one moment (Fig. 2), and Fig. 4 shows one round of the protocol. Its body holds the log's size, an RFC 6962 root over the entries' `entry_hash` values, the chain head, the evidence Merkle root and a timestamp, and the custodian signs its canonical JSON with SHA-256. `cli.py checkpoint` writes one. With `--since-size N` it also writes a *consistency proof*: a short list of hashes (9 to 14 of them for logs of 1,000 to 100,000 entries) showing that the tree over the first N entries is a prefix of the current tree, without revealing any entry ([4, Sec. 2.1.2]; verified as in [22, Sec. 2.1.4.2]).

`witness.py` is run by an independent party, such as a supervisor or opposing counsel. It has its own key and a small state file with the last size and root it cosigned for each log; it never sees the log itself. It cosigns a checkpoint only if the custodian's signature verifies and the checkpoint extends that state. The same size needs the same root, a smaller size is a rollback, and a larger size needs a consistency proof from the last size that verifies against the stored root. Otherwise it refuses, says which case it is, and changes nothing. The first time it sees a log it trusts the custodian's public key, which has to reach it by another route, and remembers it. It saves its state atomically, before it writes the cosigned file. Its signature and consistency rules come from `verifier.py`, so there is no third copy of them. Everything moves as files; there is no network.

The verifier takes the cosigned checkpoint, the log, the public keys of the trusted witnesses and a quorum. It checks the custodian's signature, counts the distinct witnesses with a valid cosignature (a cosignature signs the checkpoint body, the witness id and the time it was seen, under a tag that keeps it from being mistaken for a custodian signature), rebuilds the root over exactly the first `size` entries of the log and compares it, and, if an evidence index is given, compares the evidence root too. A log that only grew passes and reports how many entries are new. One detail needed care. A Merkle root does not encode its tree size (a consistency proof for 3 leaves also verifies as one for 4), so the size in a checkpoint is trusted only because the custodian signed it together with the root and the verifier rebuilds the root over that many entries.

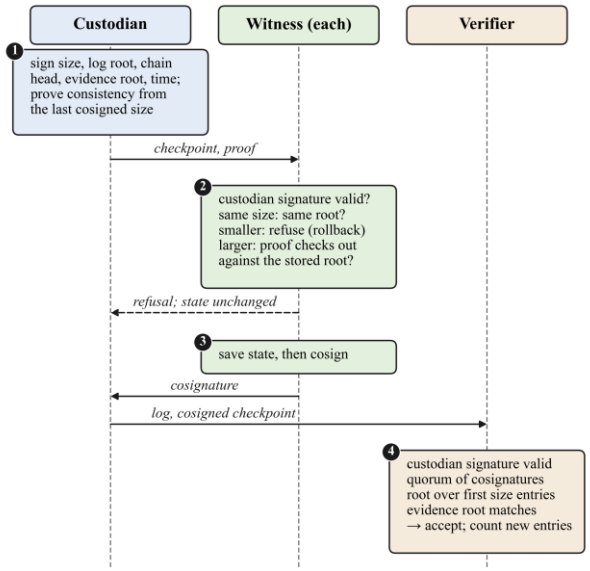


Fig. 4. One checkpoint round. A witness cosigns only a checkpoint that extends the last one it cosigned, and otherwise refuses without changing its state. The verifier needs a quorum of trusted witnesses. Every message is a file.

I. Testing

The suite has 1,686 passing tests and 2 skipped. It covers hashing (serial and pooled, including digests at the 64 KiB and 1 MiB read boundaries, buffer reuse, nested calls and concurrent threads), Merkle proofs including nested ones, tamper detection in the chain, ECDSA round trips and failures, the SQLite store and its triggers, the verifier and the anchor, the Merkle tree and its consistency proofs against the RFC text, signature schemes and legacy signatures, checkpoints, the witness's refusals and the quorum rules, multi-evidence events, and the interactive shell, including its checkpoint screens, through simulated key sequences. An earlier, smaller version of the suite (239 tests, excluding the AES-GCM storage tests) also passed under Linux (Ubuntu on WSL2); we did not repeat the full run there. Test counts show how much is exercised, not how well, and we do not report line coverage.

VI. EXPERIMENTAL SETUP AND RESULTS

All runs use synthetic evidence (random bytes from `os.urandom`), so no real case material was handled. The code paths for hashing, Merkle construction and chain verification do not depend on file content. The measurements below were taken on one Windows 11 laptop (Intel Core i9-14900HX, 24 cores and 32 logical CPUs, 32 GB of RAM, NVMe SSD) running on mains power with the Performance power plan, using Python 3.11.9. Files were freshly written before each run and so mostly sit in the page cache. We found that this machine slows any process by 2× to 3.5× after about 3 s of sustained CPU work (Section VI-C). To keep that from distorting the tables, every run starts in a fresh process after a rest of at least 4 s, and the timings of the scaling benchmark are the median of three runs. Raw JSON for every table is kept with the repository's experiment results.

A. Correctness on Clean Data

A freshly built chain and its Merkle proofs verify with the standalone verifier, and the verifier's recomputed Merkle root matches the logger's root over mixed solo-file and folder evidence. The extended harness also builds a six-entry chain that includes a multi-evidence entry and verifies it cleanly before any tampering.

B. Tamper Detection

Basic scenarios. The original four single-point corruptions were rerun on the current code. All four were detected and localised to the right entry (Table III).

Scenario	Corruption applied	Target	Detected	Reported	Reason
`flip_actor_field`	Actor changed on entry 2	2	Yes	2	`entry_hash` mismatch
`flip_action_field`	Action changed on entry 1	1	Yes	1	`entry_hash` mismatch
`corrupt_signature`	One hex digit of entry 3's signature flipped	3	Yes	3	signature failed
`break_prev_hash_link`	Entry 4's `prev_hash` altered	4	Yes	4	`prev_hash` mismatch

Table III. Basic tamper scenarios (chain verification only).

These four share a property: each changes an entry that is still in the log. They are what a hash chain is built to catch, and it is not surprising that it catches them.

Extended scenarios. We added attacks that remove or reorder entries, edit inside a multi-evidence entry, or rewrite history, and we ran each one twice: against the chain alone, and against the chain plus an anchor taken from the clean log (Table IV).

Scenario	What the attacker does	Chain only	Reported entry	With anchor
`truncate_last_two`	Deletes the newest two entries	**Missed**	–	Detected
`delete_middle_entry`	Removes entry 3, leaving a gap	Detected	4	Detected
`swap_adjacent_entries`	Swaps entries 1 and 2	Detected	2	Detected
`edit_multi_evidence_id`	Changes one id inside a multi-evidence entry	Detected	2	Detected
`rewrite_suffix_no_key`	Edits entry 3, recomputes later hashes, no key	Detected	3	Detected
`rewrite_suffix_stolen_key`	Edits entry 3, re-signs later entries with the key	**Missed**	–	Detected
`merkle_leaf_changed`	Alters one evidence digest	**Missed**	–	Detected

Table IV. Extended scenarios. "Chain only" is the custody-log check by itself; the Merkle scenario is not a custody-log property, so it can only be caught by comparing roots.

The chain alone detected 4 of 7; with the anchor, 7 of 7. For gap and reorder scenarios, the reported entry is the first one whose link fails, which is the entry *after* the removed one, not the removed entry. That is as precise as the structure allows.

The three misses are the interesting result. Tail truncation leaves a valid shorter chain. A rewrite with the signing key produces a valid chain too, because the attacker can sign anything. A changed digest in the evidence set is outside the custody log altogether. The anchor catches all three, but only because the anchor was recorded before the attack and kept somewhere the attacker could not edit. If the key is stolen and no earlier anchor exists, the design has nothing to detect the rewrite with. That is the boundary we state in Section IV-E, and these tests show it concretely.

We built every attack ourselves, on a small chain, so these results show the verifier does what we designed it to do. They are not a claim of resistance against a determined adversary.

C. Performance and Scale

Table V gives the cost of hashing, tree construction and proofs for sets of 64 KB files, in a single process, as the median of three runs, and Figs. 5a and 5b plot it. Proof generation and verification are the mean of 2,000 calls, because one call takes only microseconds and is dominated by cold-start noise.

Files	Hashing (s)	ms/file	Merkle build (s)	Proof gen (µs)	Proof verify (µs)	Steps
10	0.0008	0.08	0.000026	1.4	3.2	4
100	0.0058	0.06	0.000138	2.8	6.7	7
1,000	0.0621	0.06	0.001353	4.4	9.2	10
10,000	0.9752	0.10	0.054368	15.2	43.9	14

Table V. Single-process hashing, Merkle build and proof cost versus evidence count (64 KB files, median of three runs).

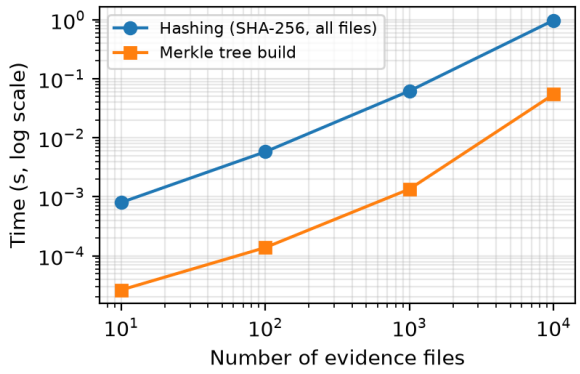


Fig. 5a. Hashing and Merkle-tree build time (log-log) versus evidence-file count.

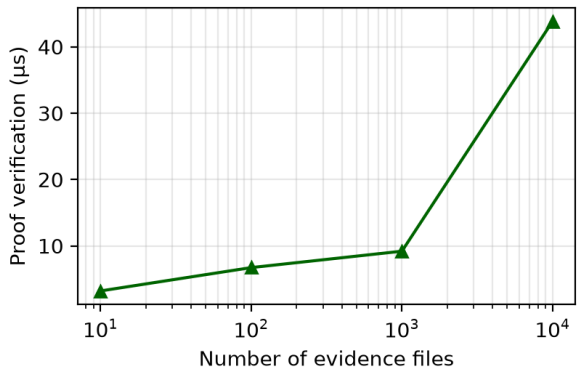


Fig. 5b. Merkle proof verification time versus evidence-file count.

Proof size grows as ceil($\log_2 n$) (4, 7, 10, 14 steps), and each step costs under 1 µs to verify, so verification time is logarithmic in the number of files. In a fresh process, the same operation takes 4.7, 7.6, 9.7 and 12.4 µs for 4, 7, 10 and 14 steps. Building the tree takes 54 ms at 10,000 leaves, so hashing, not the Merkle layer, dominates the total. (The build was 32 ms before the leaf prefix of Section IV-C, which hashes every leaf once more.)

Why the first measurement was more than six times slower. Our first run of this experiment hashed 10,000 files in 7.57 s, which is 0.76 ms per file against 0.30 ms at 1,000 files, and it repeated in three runs. We traced it to two separate causes.

A defect in our hashing code. Files of 64 KiB or more took a slower path that allocated a new 1 MiB read buffer for every file (Section V-F). Allocating it cost about 0.22 ms, against about 0.03 ms to hash 64 KiB. Table VI compares per-file cost either side of that threshold before and after the fix. Our synthetic files were exactly 64 KiB, which is the worst case, but any real file between 64 KiB and about 1 MiB paid the same cost.

File size	Before (ms/file)	After (ms/file)
4 KiB	0.029	0.029
32 KiB	0.042	0.047
65,535 bytes	0.060	0.062
65,536 bytes	0.289	0.067
128 KiB	0.323	0.101
512 KiB	0.533	0.312

Table VI. Per-file hashing cost across the 64 KiB read threshold, before and after reusing the read buffer (2,000 files per row, fresh process).

A slowdown of the machine. The cost per file also stepped up part-way through a long run, from 0.29 ms to about 0.82 ms, and a second pass over the same files was as slow as the first. CPU time equalled wall time, so the process was not waiting for the disk, and a second pass being no faster rules out the page cache. A loop of in-memory SHA-256 with no files at all showed the same step: about 35,000 hashes per second for roughly 3 s, then about 17,700. Memory allocation and plain file reads showed it as well. Windows' processor-performance counter fell from about 170–200% of the 2.2 GHz base clock to about 95% over the same interval. Pinning the process to one core, including an efficiency core, did not remove the step. A new process starts fast again, but 4 s of idle time inside the same process did not restore the speed in one test (proof verification: 11 µs in a fresh process, 39 µs after 6 s of load and 38 µs after 4 s of rest). We did not identify the mechanism; it may be a limit on the turbo boost or a power-management rule of the operating system or firmware.

The practical consequence is that, on this machine, any run longer than about 3 s proceeds at roughly half speed or less. That is why Table V uses medians, why the 10,000-file hashing time varied between 0.98 s and 1.57 s across the three runs, and why the 10,000-leaf proof timings in Table V are about three to four times what a fresh process measures. We treat the 10,000-file row as indicative. The claim that cost per file does not depend on the evidence count holds in the fast phase: 0.06 ms per file at 100 and at 1,000 files.

D. Storage

The experiment compares plaintext JSON with the earlier AES-256-GCM SQLite store.

Records	Plaintext (B)	AES-GCM SQLite (B)	Overhead
10	7,105	24,576	+246%
100	70,825	102,400	+45%
1,000	708,925	860,160	+21%

Table VII. Plaintext JSON against the earlier AES-256-GCM store.

AES-GCM itself adds 28 bytes per record (a 12-byte nonce and a 16-byte tag), which is about 4% of a roughly 700-byte record. Most of what the table shows is SQLite's page allocation: the smallest file is 24,576 bytes whatever it holds, so the percentage falls as the record count rises. Isolating the two effects would take an unencrypted SQLite baseline, which we have not run. For the store the system now uses (plain columns, no encryption), a database holding 10,001 evidence rows and one custody event was 2,670,592 bytes, about 267 bytes per row.

E. Folder Registration and Nested Proofs

Table VIII reports end-to-end folder registration, single process against the automatic worker pool, after the buffer fix. Each row is one run in a fresh process.

Folder	Mode	Wall time	Files/s	Peak memory
1,000 × 64 KB (62.5 MiB)	auto (single, under 2,000 files)	0.09 s	10,647	28 MiB
10,000 × 4 KB (39.1 MiB)	single process	0.55 s	18,192	31 MiB
10,000 × 4 KB (39.1 MiB)	auto (8 workers)	0.36 s	27,745	33 MiB
10,000 × 64 KB (625 MiB)	single process	1.01 s	9,899	32 MiB
10,000 × 64 KB (625 MiB)	auto (8 workers)	0.42 s	23,709	32 MiB

Table VIII. Folder registration, single process versus the automatic worker pool.

For the 625 MiB folder the pool is 2.4 times faster (1.01 s against 0.42 s). For 4 KB files it is 1.5 times faster, because per-file bookkeeping (walking, sorting, database writes) matters more than hashing there. Before the buffer fix, the same two runs took 8.11 s and 0.96 s, a gap of 8.4×. Most of that gap came from the slow single-process baseline, which had the per-file allocation and ran long enough to hit the machine slowdown, not from anything the pool did. We note it so that the earlier figure is not read as the pool's benefit. Peak memory stays near 30 MiB in every run, but the Python process itself takes about 25 MiB, so this shows that nothing grows noticeably, not that the footprint is tiny.

Nested-proof timings from our September run (one-off measurements, taken on the earlier ingest code and before the Merkle tree change of Section IV-C, which adds one hash per proof) were: generation 0.48 ms, 0.77 ms and 5.0 ms for folders of 10, 100 and 1,000 files; verification 8, 14 and 22 µs. Verification stays flat because the second hop only walks the main tree's depth over top-level entries, not the folder's size. The verifier's independently recomputed root matched the logger's in every trial.

F. Evidence Re-hash

The shell's *Verification → Evidence* check re-hashes every registered file against its recorded digest. We timed it on a copy of a 10,001-row database, using the same files as above (Fig. 6).

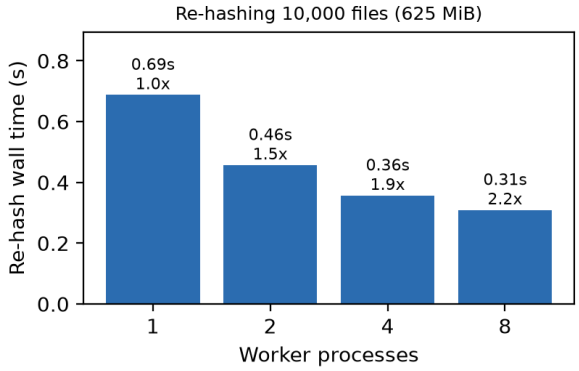


Fig. 6. Re-hash wall time by worker count, 10,000 files (625 MiB).

Time falls from 0.69 s (one worker) to 0.46 s (2), 0.36 s (4) and 0.31 s (8), a 2.2× gain. The automatic setting on this machine picked eight workers and finished in 0.32 s. Scaling is sub-linear, and in the earlier development study two workers gave only 1.35× on a machine limited to two CPUs, which is why the pool stays off below 2,000 files and is capped at eight workers. Peak memory rose from 39.4 to 42.8 MiB across the worker counts. The same check took 3.07 s with one worker and 0.95 s with eight (3.2×) before the buffer fix. These runs are warm-cache. One first run, after the files had left the operating system's cache, took 3.59 s with a single worker against 0.69 s warm: a single cold observation, not a measurement series, but a reminder that the warm figures are the best case.

G. A Larger Field Run

We also recorded larger runs on our development machine (32 CPUs, warm page cache) using a real 420,557-file, 19.6 GB folder. We did not repeat them for this paper, and we report them as recorded in the repository documentation. They were taken before the read-buffer fix of Section V-F, which makes single-process hashing faster for any file of 64 KiB or more, so the single-process baselines below would now be shorter and the ratios smaller. Registration took 43 s with eight workers, against 240–450 s single-process. Re-hashing a 419,900-file database took 36 s against 203 s single-process, and 67 s with four workers. These are warm-cache figures on a fast disk. Cold reads on a spinning disk or a network share will be slower, and the worker cap of eight was chosen from one SSD's numbers; we have not re-measured it on other storage.

H. Witness-Cosigned Checkpoints

We ran a control and 12 attacks against the real `witness.py` and `verifier.py` on synthetic data (`scripts/witness_scenarios.py`; a test pins the outcome of every scenario, so this table cannot drift from the code). The setup is an 8-entry custody log, a checkpoint of its first 6 entries cosigned by two witnesses, and a verifier that asks for a quorum of 2.

Unless a row says otherwise, the attacker holds the custodian's signing key, which is the case the hash chain alone cannot see.

Scenario	Result
Honest log, checkpoint and evidence index (control)	accepted
Rewrite entry 2, re-sign every later entry	stopped by the verifier
Delete the newest 4 entries (fewer than the checkpoint covers)	stopped by the verifier
Delete the 2 entries added after the checkpoint	**not stopped**
Rewrite an entry added after the checkpoint	**not stopped**
Show a witness another history of the same size	refused by the witness
Show a witness a smaller log	refused by the witness
Rewrite entry 2, add entries, supply a proof	refused by the witness
Attach made-up cosignatures	stopped by the verifier
Only one of the two required witnesses signed	stopped by the verifier
Remove an evidence record from the index	stopped by the verifier
Hold the custodian key and every witness key	**not stopped**
Rewrite before any witness has seen the log	**not stopped**

Table IX. Attack scenarios against a checkpoint cosigned by two witnesses. Bold rows are the design's limits.

Eight of the twelve attacks were stopped. The stolen-key rewrite and the truncation, which the chain alone missed (Table IV), are caught with no anchor, because the verifier rebuilds the root over the first six entries and the witnesses' signatures bind that root. A fork, a rollback and a rewrite followed by growth never receive a cosignature at all: the witness refuses before signing, so the attacker cannot obtain the cosigned file in the first place. A changed evidence set is caught by the evidence root inside the checkpoint.

The four that were not stopped are limits, and we want them in the table. First, entries added after the last cosigned checkpoint are not witnessed. Deleting or rewriting them passes, because nothing yet commits to them; the window closes at the next checkpoint, so how often to checkpoint is a policy decision. Second, a witness trusts a log on first sight. An attacker who rewrites history before any witness has seen the log gets the rewrite cosigned, although the honest custodian's own checkpoint is then refused as a fork, which makes the conflict visible. Third, if the attacker holds enough witness keys to meet the quorum, the cosignatures are genuine. Quorums exist to make that expensive, not impossible.

Log entries	Checkpoint (bytes)	Proof (hashes)	Tree build (ms)	Proof check (µs)	Witness cosign (ms)
1,000	504	9	1.3	12	4.0
10,000	505	12	14	14	4.1
100,000	506	14	144	17	4.3

Table X. Cost of a checkpoint, of a consistency proof from half the log size, and of a cosignature (mean of two runs, in one process).

A checkpoint is about 500 bytes whatever the size of the log, and 1,086 bytes once two witnesses have cosigned it. A proof from half the log is 9 to 14 hashes (666 to 1,010 bytes as JSON) and checks in 12 to 17 µs. The custodian pays one tree build per checkpoint (144 ms at 100,000 entries) and the verifier pays about the same to rebuild the root (146 ms). Checking the chain's own signatures costs about 1.6 ms per entry in our development measurements, roughly 160 s at 100,000 entries, so the checkpoint check is small beside it. A witness cosigns in about 4 ms because it never reads the log. These are calls inside one process; the real commands also pay Python's start-up, which we did not measure.

VII. DISCUSSION

The results support the paper's main argument with an important correction. Hash-linking, signatures and Merkle proofs do give a single custodian blockchain-style tamper evidence for edits to what is *in* the log. They do not, by themselves, show that nothing was *taken out* of it, or that history was not replaced by someone holding the key. A reader of our first draft could reasonably have concluded that they do. The extended scenarios show otherwise, and the anchor and the witnessed checkpoint are how the system answers.

The anchor is a small mechanism with a real cost. It moves part of the trust from the database to whoever holds the copy. A lab that exports an anchor after every case milestone and e-mails it to a supervisor gets a strong guarantee. A lab that never does gets the weaker, chain-only one. We think making that choice visible is better than hiding it, and the shell's `SHORTENED` warning is deliberately labelled as a convenience.

A witnessed checkpoint changes who has to be careful. With an anchor, the custodian has to keep a file safe, and a custodian who has been compromised can lose it. With witnesses, an independent party keeps the state and refuses to sign anything that is not an append-only extension of it, so a thief of the signing key cannot obtain a cosignature for a rewrite. The cost is that people have to exist and take part: a supervisor who actually runs the witness, an agreed rhythm of checkpoints, and a quorum chosen so that one lost key or one absent witness does not block the lab. We did not study that human side, and it is where we expect such a system to succeed or fail in practice.

Table XI places this work beside the approaches we found, using only what each source states.

Approach	No ledger	Signed actions	Tail cut shown	Witnesses	Own verifier
Blockchain custody systems [8]–[10], [14]	No	Yes, as transactions	By the ledger's replication	Peer nodes	Not stated
Merkle evidence containers and timestamps [19]–[21]	Yes, except [20] (Ethereum anchor)	Not stated	Not stated	Not stated	Not stated
Forensic evidence envelope [18]	Yes	No; timestamp tokens	Yes, at timestamped checkpoints	No; a timestamp authority	Not stated
Generic tamper-evident logs [3], [11]	Yes	No	Via auditors [3]	Auditors [3]	Not stated
Witnessed transparency logs [23], [25]	Yes	Signed checksums, not custody	Yes	Yes	Not stated

Approach	No ledger	Signed actions	Tail cut shown	Witnesses	Own verifier
This work	Yes	Yes	Yes, up to the last cosigned checkpoint	Yes (quorum)	Yes

Table XI. Approaches compared. "Tail cut shown" means a removed tail of the log can be detected; "Own verifier" means a checker that shares no code with the logger. "Not stated" means we did not find the property described in the paper, abstract or documentation we read; it does not mean the property is absent.

The row for this work is the only one we can vouch for. The rows above it summarise sources we read in different depth, and a reader should check them before relying on them.

The limits of the work are these.

- **Detection, not prevention.** Someone with write access can still edit the log between verifications. We make that edit visible.
- **Key compromise.** An attacker with the signing key can produce a valid chain. A witnessed checkpoint or an earlier anchor exposes a rewrite of what they cover; nothing exposes a rewrite of entries added after the last one. Threshold signatures or hardware-backed keys would help and are future work.
- **The witness window and first sight.** Entries after the last cosigned checkpoint are unprotected, and a witness trusts a log the first time it sees it (Table IX). Checkpointing at intake and at each milestone narrows both.
- **Witness trust.** A quorum of colluding or stolen witness keys defeats the scheme, and a witness that is offline cannot cosign. A witness proves only that a log extends what it saw; it says nothing about whether what it saw was true.
- **Single custodian.** If several mutually distrusting labs must share one record, consensus becomes the right tool, and this design is not a substitute.
- **Triggers are a guard, not a guarantee.** The append-only triggers can be dropped by anyone who can open the database file.
- **Evaluation scope.** The data are synthetic and the machine is one fast Windows workstation. We have one cold-cache observation but no series, no spinning-disk results, no comparison against a running blockchain baseline, and the attack scenarios are hand-built. The test machine slows any process by 2× to 3.5× after about 3 s of sustained CPU work, for a reason we did not identify, so timings of runs longer than that are indicative.
- **Duplicated logic.** Keeping the verifier independent means nested-proof and parallel re-hash logic exist twice, and the read-buffer fix of Section V-F had to be made in both copies.

A line of work we did not do, and which would strengthen the central claim, is measuring the same workload on one of the blockchain systems we cite. That would turn our argument about cost into a number.

VIII. CONCLUSION AND FUTURE WORK

We applied the transparency-log toolkit (hash-linked signed entries, Merkle trees, consistency proofs and witness cosigning, none of them new) to the chain of custody of digital evidence, built it as a working system with no ledger and no consensus layer, and evaluated what it does and does not stop. The second version added an exported anchor, witness-cosigned checkpoints, a single append-only SQLite store, multi-evidence custody events and process-pool hashing. On synthetic data, the hash chain alone detected four of seven attacks and the anchor completed the set. Witnessed checkpoints removed the dependence on the custodian keeping the anchor: eight of twelve attacks were stopped, including a stolen-key rewrite and a truncation, and the four that were not (two changes to entries added after the last checkpoint, a rewrite before the first cosignature, and theft of a whole quorum of witness keys) are stated limits. The worker pool makes registering and re-verifying a 10,000-file folder about 2.4 to 2.5 times faster, while removing a per-file buffer allocation that we found by chasing a slow benchmark mattered more still; and the repository's test suite grew to 1,686 passing tests. The more useful finding was an honest one: a hash chain on its own cannot see truncation or a key-holder's rewrite, and something outside the custodian's control has to cover them. A witness is a better such thing than a file the custodian promises to keep, but it protects only what it has already cosigned.

Future work, roughly in order of value to us: a study of how labs would actually run witnesses and choose a quorum and a checkpoint rhythm; an RFC 3161 or OpenTimestamps timestamp on each checkpoint, so that the time a witness saw it does not rest on the witness's own clock; a network witness and gossip between witnesses, and the C2SP cosignature format so that existing witnesses could be used; hardware-backed or threshold signing; a head-to-head benchmark against an existing blockchain custody system; cold-cache and HDD measurements; an unencrypted-SQLite baseline for the storage experiment; and support for more evidence containers such as E01 and dd images.

ACKNOWLEDGMENT

The authors thank Dr. Archana Kalidindi, Assistant Professor, Department of Computer Science and Engineering, KLH University (Bowrampet), for supervision and guidance.

IX. REFERENCES

[1] S. Haber and W. S. Stornetta, "How to time-stamp a digital document," *J. Cryptology*, vol. 3, no. 2, pp. 99–111, 1991.

[2] B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," *ACM Trans. Inf. Syst. Secur.*, vol. 1, no. 3, pp. 159–176, 1999.

[3] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in *Proc. 18th USENIX Security Symp.*, 2009.

[4] B. Laurie, A. Langley, and E. Kasper, "Certificate Transparency," IETF, RFC 6962, 2013.

[5] E. Syta, I. Tamas, D. Visher, D. I. Wolinsky, P. Jovanovic, L. Gasser, N. Gailly, I. Khoffi, and B. Ford, "Keeping authorities 'honest or bust' with decentralized witness cosigning," arXiv:1503.08768, 2016.

[6] S. Nath, K. Summers, J. Baek, and G.-J. Ahn, "Digital evidence chain of custody: Navigating new realities of digital forensics," in *Proc. IEEE 6th Int. Conf. Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA)*, 2024, pp. 11–20, doi: 10.1109/TPS-ISA62245.2024.00012.

[7] H. E. Patil, R. K. Kohli, S. Puri, and P. Puri, "Potential applicability of blockchain technology in the maintenance of chain of custody in forensic casework," *Egyptian J. Forensic Sci.*, vol. 14, no. 1, 2024.

[8] S. Bonomi, M. Casini, and C. Ciccotelli, "B-CoC: A blockchain-based chain of custody for evidences management in digital forensics," arXiv:1807.10359, 2018.

[9] A. H. Lone and R. N. Mir, "Forensic-chain: Blockchain based digital forensics chain of custody with PoC in Hyperledger Composer," *Digit. Investig.*, 2019.

[10] L. Ahmad, S. Khanji, F. Iqbal, and F. Kamoun, "Blockchain-based chain of custody: Towards real-time tamper-proof evidence management," in *Proc. 15th Int. Conf. Availability, Reliability and Security (ARES '20)*, ACM, 2020.

[11] M. A. Yagiz, F. Horasan, and A. H. Yurttakal, "Lightweight tamper-evident log integrity verification for IoT edge environments: A Merkle-tree pipeline with adaptive chunking," arXiv preprint, 2026.

[12] H. M. Al-Khateeb, G. Epiphaniou, and H. Daly, "Blockchain for modern digital forensics: The chain-of-custody as a distributed ledger," 2019.

[13] F. Prakash and H. Sadawarti, "Blockchain-based chain of custody: A secure digital evidence framework for digital forensics investigation," *Appl. Innov. Res.*, vol. 3, no. 1–4, 2022.

[14] A. Miller and A. Singh, "Chain of custody and evidence integrity verification using blockchain technology," in *Proc. Int. Conf. Cyber Warfare and Security (ICCWS)*, 2024.

[15] Sakshi, A. Malik, and A. K. Sharma, "Blockchain-based digital chain of custody multimedia evidence preservation framework for Internet-of-Things," *J. King Saud Univ. – Comput. Inf. Sci.*, 2023.

[16] P. G. Bradford and D. A. Ray, "Using digital chains of custody on constrained devices to verify evidence," in *Proc. IEEE Int. Conf. Intelligence and Security Informatics (ISI)*, 2007, doi: 10.1109/ISI.2007.379522.

[17] Netresearch, "ADR-034: Audit chain tip anchor," nr-vault documentation, 2026. [Online]. Available: https://docs.typo3.org/p/netresearch/nr-vault/0.15/en-us/Developer/Adr/ADR-034-AuditChainTipAnchor.html

[18] G. Longo, A. Cantelli-Forti, A. Merlo, and E. Russo, "Spectrum of liability: Forensically sound acquisition of wireless evidence for the reconstruction of maritime cyber-physical incidents," arXiv:2610.25411, 2026.

[19] J. Han, M. L. Han, S. Lee, and J. Park, "ECo-Bag: An elastic container based on Merkle tree as a universal digital evidence bag," *Forensic Sci. Int.: Digit. Investig.*, 2024.

[20] D.-O. Jaquet-Chiffelle, L. Pfeiffer, L. Brocard, E. Benoist, and N. Foukia, "Horodocs: A scalable, sustainable, robust and privacy compliant system to securely timestamp digital evidence and documents," *Forensic Sci. Int.: Digit. Investig.*, 2025, doi: 10.1016/j.fsidi.2025.301913.

[21] DFRWS USA 2020, "Revisiting the linear hash," conference presentation slides, 2020. [Online]. Available: https://dfrws.org/wp-content/uploads/2020/06/2020_USA_pres-revisiting_the_linear_hash-1.pdf

[22] B. Laurie, E. Messeri, and R. Stradling, "Certificate Transparency version 2.0," IETF, RFC 9162 (Experimental; obsoletes RFC 6962), 2021. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9162

[23] C2SP, "Transparency log cosignatures (tlog-cosignature, v1.0.0)," specification. [Online]. Available: https://github.com/C2SP/C2SP/blob/tlog-cosignature/v1.0.0/tlog-cosignature.md

[24] transparency.dev, "Can I get a witness (network)?," blog post. [Online]. Available: https://blog.transparency.dev/can-i-get-a-witness-network

[25] Sigsum, "Sigsum: Transparency for signed checksums," project documentation. [Online]. Available: https://git.sigsum.org/log-go/about/?h=v0.2.0

[26] H. Birkholz, A. Delignat-Lavaud, C. Fournet, Y. Deshpande, and S. Lasker, "An architecture for trustworthy and transparent digital supply chains," IETF Internet-Draft draft-ietf-scitt-architecture-09, work in progress, 2024. [Online]. Available: https://datatracker.ietf.org/doc/html/draft-ietf-scitt-architecture-09